GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

数据流驱动的机器人导航：DORA-rs架构下从感知到控制的实现

肖文程 石家庄铁道大学 博士研究生

2026.10.16



CONTENTS

Part 01 问题与定位

Part 02 数据流机制

Part 03 导航闭环实现

Part 04 工程验证



从社区、框架到工程实现-以室内巡检为载体的工程实践

报告一

社区共建与具身实践

为什么做 · 谁在做

- DORA-rs 开源生态
- 具身智能探索
- 机器人端到端落地

报告二

基于DORA的三维导航框架

做什么 · 能做什么

- 3D激光雷达点云
- 空间推理与建图
- 轨迹规划框架

本报告

感知到控制（2D室内巡检）

怎么跑 · 如何验证

- ✓ 2D激光 YAML 拓扑
- ✓ 数据通路与共享内存
- ✓ 性能与调试

聚焦“2D室内巡检”工程落地，追踪真实导航数据从感知到控制的完整生命周期。

导航的起点：SLAM 是什么

SLAM · Simultaneous Localization and Mapping

核心：在未知环境中，同时解决“我在哪”与“地图长什么样”的问题！

定位 · 我在哪?

在地图上估测机器人的位置坐标

建图 · 地图长什么样?

根据传感器结果构建或修正地图

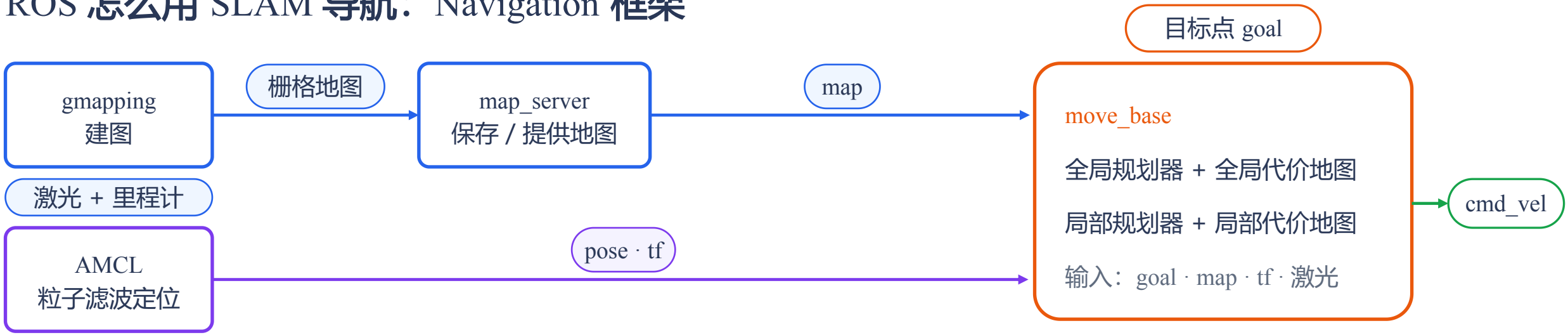
前提：机器人需携带激光雷达等测距设备，获取环境深度信息

gmapping **2D激光SLAM**

- 基于激光雷达的 SLAM 功能包
- 输入激光雷达+里程计信息
- Rao-Blackwellized 粒子滤波算法/图优化
- 输出二维栅格地图
- 地图话题：nav_msgs/OccupancyGrid

▸ 规划路径—传统ROS 用 Navigation 框架完成这件事。

ROS 怎么用 SLAM 导航：Navigation 框架

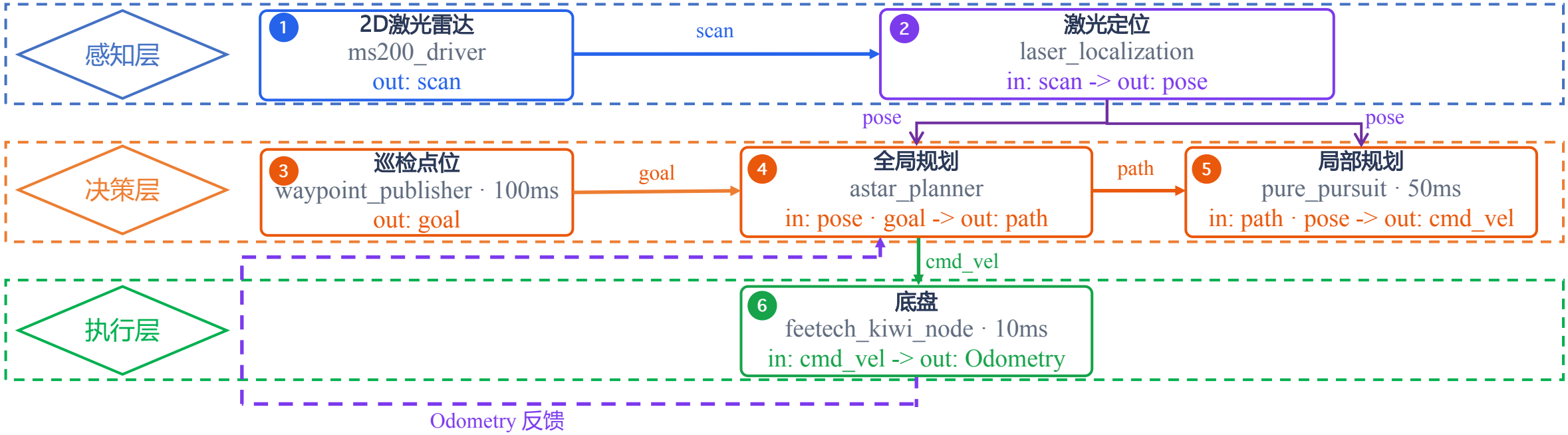


使用流程：**roslaunch 建图 → map_saver 保存地图 → roslaunch 导航（AMCL + move_base）**

环节	ROS 方案	DORA_NAV
定位与地图	AMCL 粒子滤波 + map_server 静态地图	hdl_localization 点云配准 · 消费先验地图
规划	move_base 内置全局 + 局部规划器	astar + pure_pursuit / dwa 独立节点
代价地图	costmap 插件常驻运行	DWA 节点内按需构建（6m×6m）
通信	topic 发布订阅 + master	数据流端口 + 共享内存零拷贝

▸ 差异不在算法，而在结构：DORA 把它拆成数据流上可组合的独立节点。

导航不是一个算法，而是一条实时数据链路



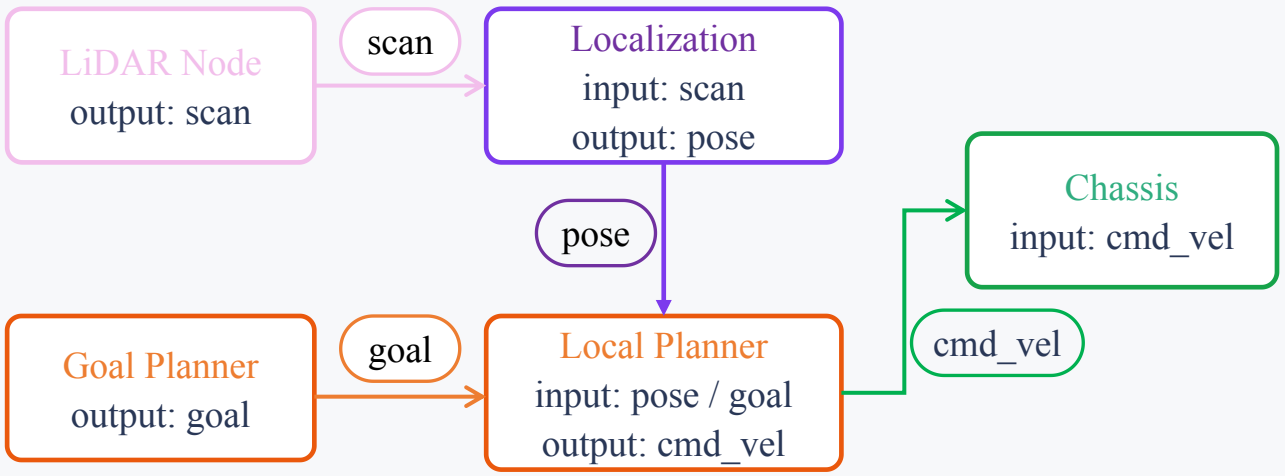
1 数据如何连接?

2 数据如何高效传输?

3 延迟与故障如何发现?

- 核心问题：从传感器采样到执行器接收指令，中间究竟发生了什么?

节点负责计算，数据流负责连接



DORA 数据流拓扑图

DORA 概念	→	导航中的含义
Node	→	算法程序
Input / Output	→	数据接口
Dataflow YAML	→	系统连接说明书
Arrow	→	统一数据表示（列式内存）

节点可用 Rust / Python / C / C++ 编写；Coordinator 与 Daemon 负责启动、监控和停止。

- 算法写在节点中；连接、生命周期与数据依赖写在 Dataflow YAML 中。

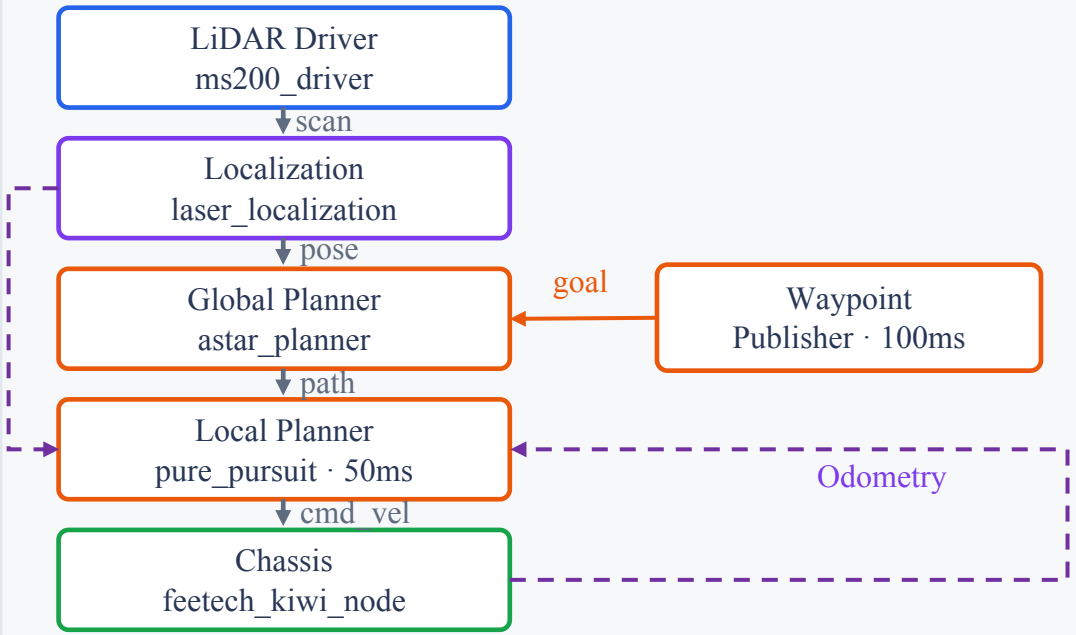
从声明式 YAML 到可运行导航图

dataflow.yml

```yaml
nodes:
  - id: ms200_driver       # 2D 激光雷达
    outputs:  [scan]
  - id:  laser_localization # 激光地图匹配
    inputs: { scan: ms200_driver/scan }
    outputs: [pose]
  - id: astar_planner      # 全局规划
    inputs: { pose: laser_localization/pose,
              goal: waypoint_publisher/goal }
    outputs: [path]
  - id: pure_pursuit       # 局部规划 · 50ms
    inputs: { pose: laser_localization/pose,   timer: dora/timer/millis/50,
              goal: waypoint_publisher/goal,
              path: astar_planner/path,
              Odometry: feetech_kiwi_node/Odometry }
    outputs: [cmd_vel]
```

timer: dora/timer/millis/50 → 20 Hz 周期触发局部规划，不依赖上游频率

运行时拓扑（Coordinator解析后）



Coordinator 解析 YAML → 启动节点、建立连接；改端口 = 改 YAML 重启

- YAML 描述的不是单个算法参数，而是整个系统的数据依赖与运行结构。

Arrow 与共享内存：减少数据搬运，而不是「零成本」。

本机大消息	点云 · 图像 · 栅格地图	> 4 KiB	共享内存
本机小消息	位姿 · 状态 · 速度指令	< 4 KiB	轻量消息通道
跨设备通信	远程传感器 · 分布式节点	Network	Zenoh 数据平面

传统路径：对象 → 序列化 → 复制 → 传输 → 反序列化 → 对象 ✕

DORA 大消息路径：Arrow 数据 → 共享内存 → 接收端直接读取 √

- 「零拷贝」有适用边界；对导航而言，真正的价值是点云与图像高效流动、接口保持统一。

一帧雷达数据如何变成速度指令



ADORA-NANO

移动机器人智能体黑客松竞赛

2D 激光 + 深度相机（室内工业场景巡检）

端到端延迟

T端到端 = T采样 + T定位 + T规划 + T传输 + T执行

一环慢，全线慢

平均值 vs 分布

只看平均值，看不到最坏情况

同时关注 P50 · P95 · P99

T0 T1 T2 T3 T4 T5

雷达采样 → scan 发布 → 定位 → 局部规划 → cmd_vel 发布 → 底盘接收

▸ 机器人能否及时避障，取决于整条链路，而不只是规划器计算耗时。

以 DWA 为例：如何封装一个局部规划节点

实际端口 · dwa_planner_dora

timer　周期触发 · 50ms

path　全局路径 astar

pose　当前位姿

scan 2D激光·直接消费

Odometry　当前速度

实际输出

cmd_vel　JSON {linear.x, angular.z}

dwa_trajectories　预测轨迹 · 可视化

节点内流水线

激光数据预处理（过滤·降采样）

↓

构建局部代价地图 6m×6m

↓

路径变换到机器人坐标系

↓

动态窗口采样角速度

↓

避障 + 朝向加权评分

↓

输出最优 (v, w)

评分函数

score = w_obs·障碍得分 + w_head·朝向得分

碰撞轨迹得分 −∞，直接淘汰

```
# DWAConfig 默认值
fixed_v: 0.2          # m/s
max_w: 2.0            # rad/s
max_accel_w: 5.0      # rad/s²
w_samples: 20
weight_obstacle: 2.0
weight_heading: 0.6
```

DWA 节点已在 DORA_NAV 完整实现；run.yml 默认启用 pure_pursuit，DWA 为可替换的局部规划器。

▸ DWA 通过标准端口接入数据流；激光 scan 直接消费，代价地图在节点内构建。

导航系统怕的不是丢掉旧数据，而是执行过期数据

错误：队列积压 → 处理过期数据



若旧帧不被丢弃：队列积压，决策滞后

实时：丢弃旧帧，始终处理最新状态

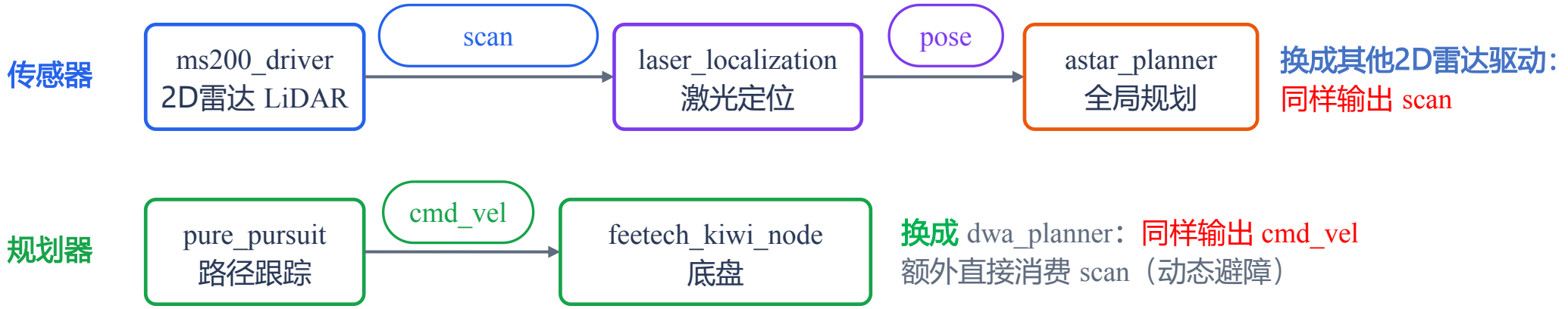


旧帧直接丢弃，永远处理最新数据

数据类型	优先目标	推荐策略
雷达点云	新鲜度	小队列 · drop_oldest（默认）
位姿 / Odometry	最新状态	覆盖式更新 · 防止积压
目标点 goal	任务一致性	低频定时发布 · 100ms
规划输出 cmd_vel	实时性	50ms 周期 · 到期即换

▸ 队列越大不代表越可靠；对传感器数据，处理过期帧比丢帧更危险。

算法可以替换，数据契约必须稳定



数据契约：scan = [ts 8B + N×8B] · cmd_vel = {linear.x, angular.z} · 头 {stamp, frame_id, seq}

Schema 一致

坐标系一致

时间戳语义一致

更新频率满足要求

- 可组合性的关键不是「改两行 YAML」，而是上下游共同遵守稳定的数据契约。

把「机器人为什么这样动」变成可追踪问题

工具链

dora graph	— 静态拓扑检查
dora list	— 运行状态
dora logs	— 节点日志
dora topic	— 话题与消息观测
.drec	— 录制与回放
OpenTelemetry	— 链路追踪

可复现调试闭环

1. 真机采集一次
2. 保存 .drec
3. 重复回放
4. 更换规划参数
5. 比较轨迹与延迟

▸ 同一份传感器输入可重复验证不同算法，使性能比较具有可复现性。

GOSIM

GOSIM Shenzhen 2026

Thanks!

